

ACTIVATION SPARSIFICATION IN TRANSFORMERS: THRESHOLD PLACEMENT, PRESSURE PLACEMENT, AND SPARSE EXECUTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation sparsity can accelerate language-model inference when specialized kernels skip computation associated with zero activations. Existing methods obtain sparsity through different combinations of training-time pressure, thresholding, and placement, but their relative and combined effects have not been isolated under a common setup. As a result, it remains unclear which interventions drive the gains, when they are complementary or redundant, and how much model-wide computation they actually expose to sparsification. We study these interventions under matched pretraining conditions across Pythia models. To measure their computational potential, we define model-wide activation sparsity $\mathcal{S}_{\text{model}}$ as the fraction of forward matrix-multiplication products with a zero activation operand, together with an architecture-dependent sparsity ceiling $\mathcal{S}_{\text{model}}^{\max}$. Pressure placement changes the quality–sparsity trade-off: in the 14M sweep, h-only pressure gives favorable moderate-threshold operating points, while broad seven-site pressure becomes useful for extreme sparsity. The broad-pressure recipes reach 27.48%, 40.60%, and 80.62% $\mathcal{S}_{\text{model}}$ at 14M, 70M, and 410M parameters, corresponding to 0.918, 0.822, and 0.924 of their respective sparsity ceilings. Selected trained recipes also outperform uniform post-hoc thresholding at high sparsity across all model sizes. For 35 measured 14M checkpoints on an RTX5090, structured projection work bypass is more strongly associated with projection-skipping gain ($R^2 = 0.941$) than scalar projection sparsity ($R^2 = 0.425$); larger logical sparsity need not imply lower optimized latency. These results show that activation sparsification is governed by the interaction between how zeros are induced, where they are placed, and how much of that zero structure can be exploited by sparse execution.

1 INTRODUCTION

Activation sparsity can improve language-model inference efficiency when specialized kernels skip computations associated with zero activations. Recent work has shown that sparsity can be exposed in pretrained models through post-hoc thresholding or induced during training, in both cases with measured inference speedups (Liu et al., 2024; Cetin et al., 2026). These results raise two practical design questions: where should zeros be exposed and, independently, where should sparsity pressure be applied? Which resulting zero structure can kernels exploit?

Existing approaches induce activation sparsity in different ways. TEAL applies post-hoc thresholding to pretrained models, ProSparse uses L1 regularization on feed-forward (FFN) activations during training, and Q-Sparse applies top-k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). While each approach is effective in its own setting, it remains unclear how they compare under a common setup and how their effects interact when combined.

In this work, we study the effects of (i) activation L1 pressure, and (ii) *thresholding*, i.e. mapping small activations exactly to zero. While (i) penalizes big activation magnitudes and force its distribution to concentrate around small values, (ii) is the non-linearity that effectively maps them to zero.

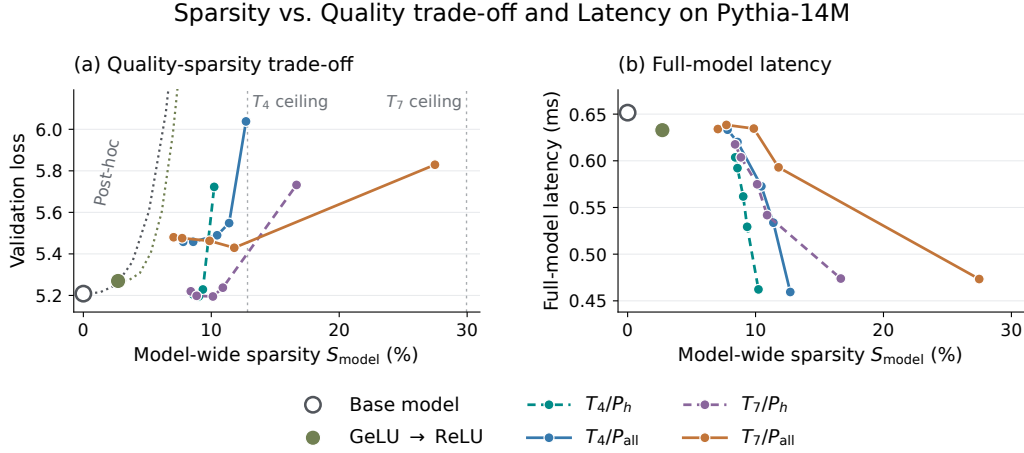


Figure 1: **Quality and execution depend on threshold and pressure placement.** Both panels show the same 22 Pythia-14M checkpoints: the base and GeLU-to-ReLU controls, and four recipes combining four-/seven-site thresholding with OL1 pressure at h or all thresholded sites. **(a)** Final-checkpoint validation loss versus model-wide sparsity. Annotated dotted paths apply post-hoc thresholding at a, m, h, z to the controls; eight high-loss evaluations are outside this view (Figure 13). Vertical guides mark analytic selected-site sparsity ceilings, not runtime gains. **(b)** Full-model K050 latency for the same trained models; the clipped controls have no matched timing measurements. Latency is the geometric mean of 1,344 host timings (64 inputs, seven passes, three processes) on RTX5090, with BF16, batch one, 2,048-token full-sequence inference and all 50,304 logits. Loss and count-pooled sparsity use FP16 validation over 338 complete blocks from 500 documents, excluding the 1,444-token tail. Seven-site h -only timings use a different physical GPU/host session; small cross-session differences are descriptive. Lines connect separately trained $\kappa \in \{0, .01, .05, .1, .5\}$ settings, not training trajectories or attainable interpolations. Each setting has one final checkpoint and one training seed.

Where to place (i) or/and (ii) is, however, combinatorial. We evaluate the individual and combined effect in different sites inside the same transformer block.

To measure their effects, rather than report sparsity locally at individual activation sites, we define model-wide activation sparsity, S_{model} , as the fraction of forward matrix-multiplication products with a zero activation operand. We additionally derive an architecture-dependent maximum sparsity that can be achieved through the intervention set ($S_{\text{model}}^{\text{max}}$).

We pretrain Pythia-family models from scratch for controlled comparisons across model sizes (Biderman et al., 2023). Across 40 conditions at 14M, we separate threshold placement from pressure placement. Figure 1 summarizes the quality and execution trade-offs for the 22 base, ReLU, and multisite-pressure checkpoints. Pressure only at the FFN hidden site gives favorable moderate-threshold trade-offs. Holding that pressure fixed, broader thresholding adds 6.44 percentage points of model-wide sparsity for +0.0094 loss at the highest threshold. Expanding pressure to all seven sites then adds another 10.82 points for +0.0974 loss, reaching 27.48% sparsity near the 29.95% ceiling. Selected broad-pressure recipes recur at 70M and 410M; h -only transfer remains untested.

The execution study establishes a third distinction: logical sparsity and kernel-exploitable structure are different. Across the 35-checkpoint runtime cohort, projection MMA bypass better explains the measured projection-skipping gain than scalar projection sparsity. We use GPT-6-Astra (OpenAI, n.d.) to develop the specialized implementation, illustrating how coding agents can support this evaluation (Luo et al., 2026).

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes small activations at inference-time of a pre-trained model. While CATS thresholds small-magnitude activations within FFNs (Lee et al., 2024), TEAL extends it to attention (Liu et al., 2024). These studies motivate training-time interventions to adapt activation distributions to sparse computation and improve the trade-off between quality and sparsity.

ReLU Strikes Back studies the effect of activations functions in language models (Mirzadeh et al., 2023), showing clear sparsity gains on the use of ReLUs. ProSparse combines progressive ℓ_1 pressure and thresholding at FFNs during continued training (Song et al., 2024). It does not, however, extend its interventions to attention and restrict the analysis to local FFN sparsity. Cetin et al. (2026) extends ProSparse setup to pre-training and propose specialized kernels. These studies show that training can reshape the quality–sparsity trade-off, but they’re restricted to a single local intervention at FFN sites.

Q-Sparse extend sparsification beyond FFNs. It applies top- k selection to both attention and FFN projection inputs (Wang et al., 2024), but with no ℓ_1 pressure. Its top- k operation introduces, however, additional overhead due to the sorting operation. Spark Transformer tries to address it with statistical top- k (You et al., 2025), but we take a simpler approach and use a constant thresholding non-linearity. We argue that when the intervention is introduced during training, the network can adapt itself. Additionally, we evaluate the complementary effect of regularization pressure on both attention and FFNs.

2.2 INFERENCE SPEEDUP

Turning activation sparsity into speedup requires a specialized kernel adapted to the model architecture. TEAL accelerates batch-one autoregressive decoding, reporting up to $1.8\times$ speedup at 50% sparsity (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication with their TwELL kernel, and report $1.3\times$ speedup for batch inference. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory access must compensate the additional overhead introduced by the sparse operations. This development has been increasingly accelerated by the use of coding agents, through iterative kernel development with execution and profiling feedback (Ouyang et al., 2025; Luo et al., 2026). We take a related approach and use GPT-6 Astra to develop a specialized kernel based on our training interventions (OpenAI, n.d.).

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

A sparsification intervention specifies (i) where to threshold activations (thresholding sites $\mathcal{N}$), (ii) the thresholding function $G(\cdot)$, and (iii) where to apply activation pressure (pressure sites $\mathcal{P}$). The sets $\mathcal{N}$ and $\mathcal{P}$ need not coincide, and different sites may use different functions G . These choices give a combinatorial set of interventions during training.

We write T_i/P_j to distinguish threshold and pressure scope: T_4 thresholds a, m, h, z , and T_7 additionally thresholds q, k, v ; P_0 applies no pressure, P_h applies OL1 only at h , and P_4/P_7 apply OL1 at all four/seven sites. In figure labels, these are “4/7-site,” “+ OL1(h),” and “+ OL1(all).”

To avoid the overhead of exact top- k selection (Wang et al., 2024; You et al., 2025), we extend ReLU to one-sided and symmetric thresholding: $G_{+, \kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm, \kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. The threshold κ controls thresholding strength and is fixed within each training condition. Appendix A gives boundary and derivative conventions.

Figure 2 locates the intervention sites on the Pythia architecture: a and m are the normalized inputs to the attention and FFN branches; h is the FFN hidden activation; z is the concatenated attention context before output projection; and q, k, v are the query, key and value operands, with q, k taken

after RoPE. The attention probabilities are denoted by p . We apply $G_{+, \kappa}$ at selected sites among a, m, h, z and $G_{\pm, \kappa}$ at q, k, v . At h , the selected nonlinearity replaces GELU.

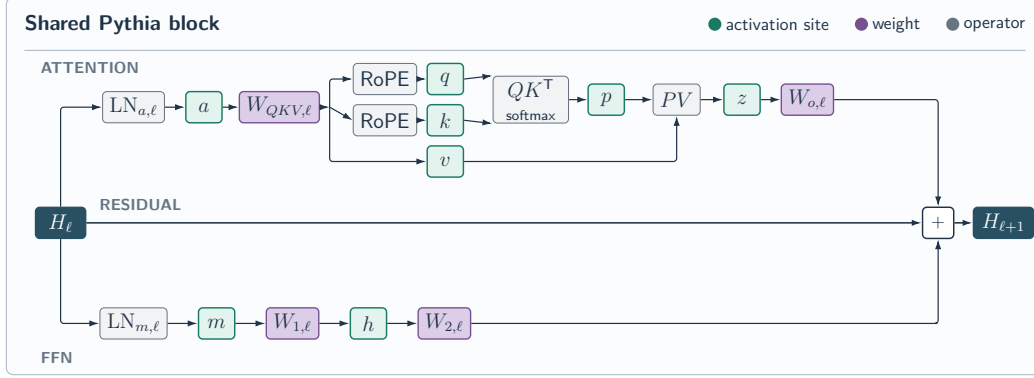


Figure 2: **Intervention sites in a shared Pythia block.** Attention and FFN branches use separate LayerNorms and join the residual path. Green nodes identify activation sites; purple nodes identify learned weights. Table 1 specifies thresholding and pressure assignments.

For optimization pressure, $\mathcal{L}_1$ averages mean absolute activations across targeted sites and layers, after any thresholding. *Naive LI* (LIN) minimizes $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$, introducing a weight λ that affects sparsity and quality (Song et al., 2024; Cetin et al., 2026). To reduce this additional tuning dimension, we propose *orthogonal LI* (OL1), inspired by gradient-conflict projection methods (Yu et al., 2020; Hsieh et al., 2024). OL1 chooses the step that maximizes alignment with the preconditioned pressure direction, within a unit norm budget relative to the adaptive Adam direction and without opposing it. Once the norm budget binds, further increasing λ leaves the ideal correction unchanged at a fixed optimizer state. To keep the multisite intervention study tractable, we use a common $\lambda = 1$ and budget $b = 1$, and directly measure whether this budget is active during training. Section 4.2 shows that budget saturation is strongly target-set dependent rather than universal. Appendix A.2 gives the formulation.

3.2 SPARSITY MEASURE AND ARCHITECTURAL REACH

Activation sparsity is the fraction of zero activations at a site. To report *model-wide sparsity*, we define $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in the Pythia-block matrix products. N_{model} counts all scalar multiplications within those blocks and the final output projection. We pool validation counts, excluding masked attention pairs (Appendix B).

To quantify intervention potential a priori, we define the *model-wide sparsity ceiling* as $\mathcal{S}_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$. It measures the fraction of scalar multiplications guaranteed a zero operand when all selected sites are 100% sparse. It depends on architecture, sites, and sequence length. Table 1 illustrates the higher ceilings of larger models at fixed sequence length. Natural zeros outside this reach can make $\mathcal{S}_{\text{model}}$ exceed $\mathcal{S}_{\text{model}}^{\text{max}}$. Appendix B.2 gives the precise accounting.

4 EXPERIMENTAL STUDY

We pretrain randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023). To fit our compute budget, we adapt Pythia’s training recipe to MiniPile (Kaddour, 2023), using 1.493 billion input tokens per model. We conduct extensive comparisons at 14M and promote the sparsest recipes to 70M and 410M, alongside their baselines. We measure validation cross-entropy loss and $\mathcal{S}_{\text{model}}$ over all complete sequences from MiniPile’s validation documents. Table 1 maps the ten recipes onto the sites in Figure 2.

The experimental setup contains 40 training runs at 14M and 12 at each larger model size, for 64 endpoints. Within each size, runs share initialization, data order and training budget. We vary

Table 1: **Sparsification recipes and architectural reach.** Ceilings are $100 \cdot S_{\text{model}}^{\max} \%$ at $T = 2,048$. A1-H recipes use ReLU at h ; T_4/T_7 use the thresholding functions in Section 3.1. A1-H-L1 uses L1N; P_h , P_4 and P_7 use OL1. All recipes are evaluated at 14M; 70M and 410M use A0, A1-H, T_4/P_4 and T_7/P_7 .

Recipe	Thresholding sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Ceiling (%)		
			14M	70M	410M
A0	$\emptyset$	$\emptyset$	0.00	0.00	0.00
A1-H	h	$\emptyset$	4.28	12.35	24.93
A1-H-L1	h	h	4.28	12.35	24.93
A1-H-OL1	h	h	4.28	12.35	24.93
T_4/P_0	a, m, h, z	$\emptyset$	12.83	37.06	74.78
T_4/P_h	a, m, h, z	h	12.83	37.06	74.78
T_4/P_4	a, m, h, z	a, m, h, z	12.83	37.06	74.78
T_7/P_0	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
T_7/P_h	a, m, h, z, q, k, v	h	29.95	49.42	87.25
T_7/P_7	a, m, h, z, q, k, v	a, m, h, z, q, k, v	29.95	49.42	87.25

local pressure for A1-H using L1N and OL1 with $\lambda \in \{0.05, 0.1, 0.5, 1\}$. At 14M, we sweep $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$ under both threshold scopes, with no pressure, h-only OL1, or OL1 at all thresholded sites. Multisite OL1 uses $\lambda = b = 1$. The larger sizes retain only the broad-pressure sweeps. Figure 4 characterizes the resulting cap activity at 14M. These sweeps test which interventions yield the largest sparsification gains. Appendix C.4 details the training settings.

To compare the interventions with post-hoc thresholding, we apply uniform percentile magnitude-thresholding targets $p = 0, 0.1, \dots, 0.9$ at a, m, h, z as a TEAL-style baseline (Liu et al., 2024). Site-layer thresholds are calibrated separately on ten training blocks.

4.1 QUALITY-SPARSITY OPERATING REGIMES

Narrow pressure is favorable at moderate thresholds, while broad seven-site pressure exposes the highest sparsity (Figure 1). For both threshold scopes at $\kappa \leq 0.1$, h-only OL1 lowers validation loss relative to no pressure by 0.18–0.26. At $\kappa = 0.05$, T_7/P_h reaches 10.126% model-wide sparsity at loss 5.19496, compared with A0’s 5.2086. This is comparable loss at the evaluated seed, not evidence of improved generalization. Broad seven-site pressure is not uniformly preferable: at this threshold, T_7/P_7 has both higher loss (5.46280) and lower sparsity (9.863%).

At $\kappa = 0.5$, the trade-off changes. Broad seven-site pressure reaches 27.483% model-wide sparsity, compared with 16.664% under h-only pressure, at losses 5.82941 and 5.73205. This high-sparsity operating point uses 91.8% of the seven-site ceiling, with a quality cost relative to A0. It is distinct from the moderate-threshold regime in which narrow pressure is favorable.

In the local ReLU sweep, OL1 is not uniformly better than naive L1: their validation-loss differences are small, and the ordering reverses at $\lambda = 1$ (Appendix Table 6). Full trained endpoints appear in Table 5. Post-hoc thresholding at a, m, h, z has four-site reach; it does not share the seven-site recipe’s attention coverage.

4.2 PAIRED EFFECTS OF PRESSURE PLACEMENT

We hold threshold scope and κ fixed and separate two pressure increments: $P_0 \rightarrow P_h$, then $P_h \rightarrow P_{\text{all}}$, where P_{all} is P_4 or P_7 (Figure 3). The first increment supplies the large moderate-threshold quality improvement. Broadening pressure then raises loss at every tested threshold, but its sparsity increment depends strongly on the operating point. For seven-site thresholding it is negative through $\kappa = 0.05$, then rises to 10.819 pp at $\kappa = 0.5$ for +0.0974 loss. At the same high threshold, broadening four-site pressure adds 2.486 pp for +0.3153 loss.

At $\kappa = 0.5$, expanding seven-site pressure from h to all sites adds 10.82 pp sparsity for +0.0974 validation loss. These contrasts separate pressure scope from threshold scope. They do not isolate

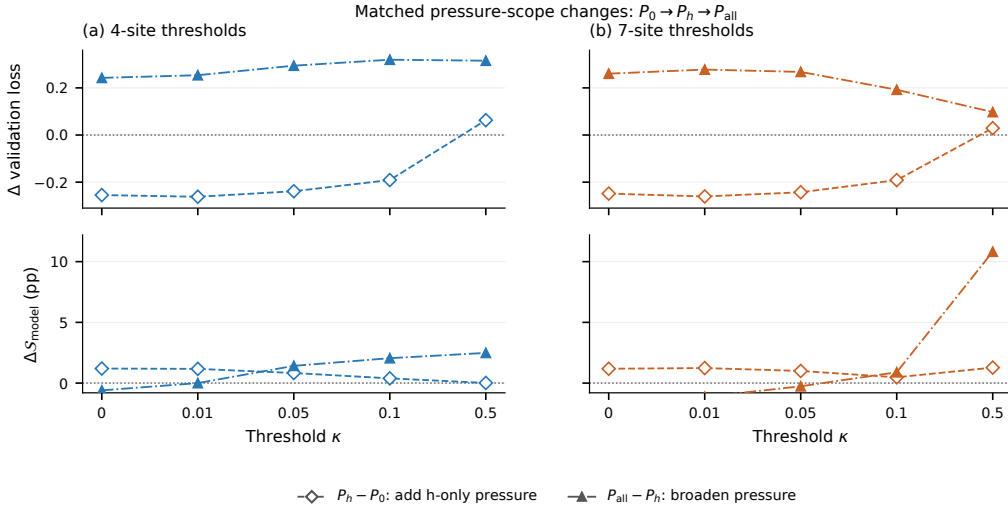


Figure 3: **The marginal effect of pressure depends on its scope and the threshold.** Columns hold four- or seven-site thresholding fixed. Each point subtracts the reference at the same κ : diamonds show $P_h - P_0$, triangles show $P_{\text{all}} - P_h$. The top row reports validation-loss change and the bottom row model-wide sparsity change in percentage points. Lines join separately trained, matched settings, not training trajectories. Each condition has one seed, with matched initialization, data order and training budget.

Q/K/V pressure: T_7/P_4 is absent, so $P_h \rightarrow P_7$ adds a, m, z, q, k, v jointly. It also changes the equal-tensor normalization of the pressure objective. Appendix Table 7 gives every increment.

Pressure scope also changes the realized optimization regime (Figure 4). All 20 pressured multisite runs retain 712 optimizer-step records. We measure the task-relative opposing component $\rho_{\text{opp}} = \max(0, -\langle u, w \rangle) / (\|u\|_2^2 + \epsilon)$ in the global AdamW-preconditioned geometry. H-only pressure is capped on 0.25% of steps under either threshold scope; broad four-site pressure is capped on 0.51%, versus 96.99% for broad seven-site pressure. The pooled median pre-cap ratio r/b is 0.102 and 0.086 for four/seven-site h-only pressure, compared with 0.253 and 57.49 for broad pressure. These logs establish different magnitude and budget regimes; they do not identify the cause of the validation-loss differences.

4.3 BROADER THRESHOLD PLACEMENT UNDER MATCHED PRESSURE SCOPE

We compare seven- with four-site thresholding at the same κ , first without pressure and then with pressure fixed at h (Table 2). **At $\kappa = 0.5$, broader thresholding under h-only pressure adds 6.44 pp model-wide sparsity for +0.0094 loss.** The corresponding pressure-free increment is 5.17 pp for +0.0432 loss. Holding pressure scope fixed gives a cleaner threshold-placement comparison than changing thresholding and pressure together. The small nonzero differences at $\kappa = 0$, where the additional Q/K/V gates are identities, remain visible.

4.4 PRESSURE PLACEMENT RESHAPES LOCAL SPARSITY

The activation response depends on where pressure is applied (Figure 5). We report m and h separately: pooling them weights the wider h tensors four times as heavily and can hide changes at the FFN input. At $\kappa = 0.5$, four-site pressure expansion from P_h to P_4 raises the m zero fraction from 66.44% to 96.77%, while h remains about 99.94% zero. Under seven-site thresholding, expanding P_h to P_7 changes m from 67.41% to 68.70%, but pooled Q/K/V zeros from 29.52% to 95.60%.

The operation decomposition explains why the last comparison produces much more model-wide sparsity. Broad seven-site pressure exposes zeros in QK/PV , which together contribute 16.77 pp,

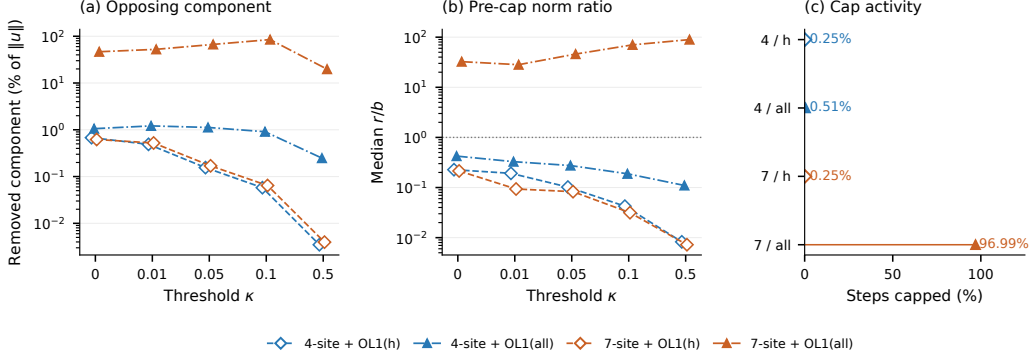


Figure 4: **H-only and broad pressure operate in different norm-budget regimes.** (a) Median removed opposing component, $100\rho_{\text{opp}}$; (b) median pre-cap ratio r/b at $\lambda = b = 1$. Each point summarizes 712 optimizer steps at a fixed threshold, not seed uncertainty. Color denotes threshold scope; diamonds/dashed lines denote h-only pressure and triangles/dash-dot lines denote broad pressure. (c) Cap-active fractions pool 3,560 steps per recipe and use the implemented condition $s < 1$. The geometry precedes group learning rates and decoupled weight decay, so it does not guarantee task-loss preservation. The h-only and broad-pressure comparison changes the objective composition and normalization.

Table 2: **Broader threshold placement under fixed pressure scope.** Entries are seven-site minus four-site at matched κ , with no pressure (P_0) or h-only OL1 (P_h). Positive sparsity differences indicate more logical zero-product opportunity; loss differences are single-seed observations.

κ	P_0 : no pressure		P_h : h-only OL1	
	ΔL	$\Delta \mathcal{S}_{\text{model}}$ (pp)	ΔL	$\Delta \mathcal{S}_{\text{model}}$ (pp)
0	-0.0021	+0.006	+0.0041	-0.010
0.01	-0.0077	+0.204	-0.0064	+0.270
0.05	+0.0038	+0.921	-0.0006	+1.090
0.1	+0.0090	+1.472	+0.0087	+1.560
0.5	+0.0432	+5.171	+0.0094	+6.436

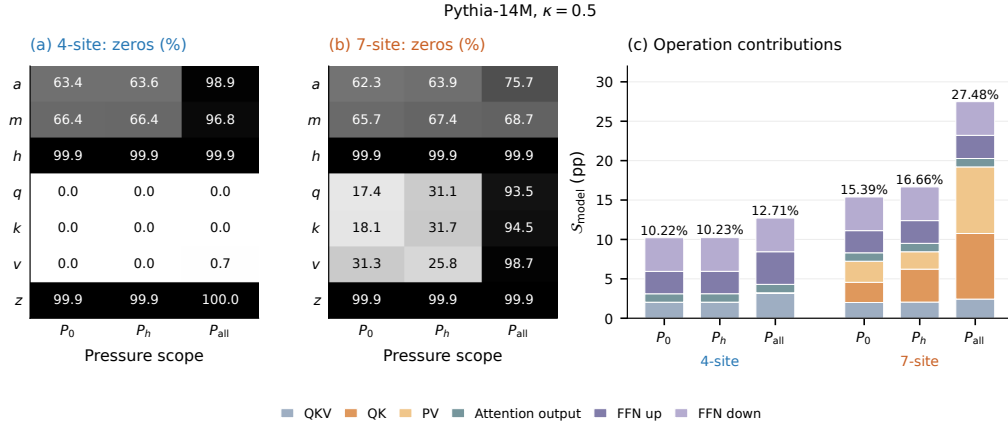


Figure 5: **Pressure placement changes activation structure and its computational effect.** All six multisite recipes at $\kappa = 0.5$, Pythia-14M. **(a,b)** Exact-zero percentages at each site, pooled from integer counts across validation and layers; m and h are separate. P_{all} means P_4 or P_7 . Rounded 0.0/100.0 entries need not be exact endpoints. **(c)** Operation contributions use the common full-model product denominator; totals appear above each bar. Broad four-site pressure mainly increases projection-side opportunity, whereas broad seven-site pressure exposes large QK/PV contributions. These are logical zero-operand products, not measured runtime savings.

compared with 0.06 pp under broad four-site pressure. Thus, even with fewer zeros at m , T_7/P_7 reaches 27.48% model-wide sparsity versus 12.71% for T_4/P_4 . Site fractions and the amount of computation consuming them jointly determine $\mathcal{S}_{model}$; the bars do not measure runtime savings.

At $\kappa = 0.05$, h-only pressure raises h sparsity from about 81% to 98% under both threshold scopes while leaving m near 52%. Under seven-site thresholding, pooled $Q/K/V$ zeros also change from 3.62% to 4.20%, despite those sites receiving no direct pressure. We call changes at nonpressured sites a *nonlocal response*; the matched $P_0 \rightarrow P_h$ comparison holds threshold scope fixed but does not identify the internal pathway. Appendix Figure 9 and Figure 10 give the moderate-threshold and site-by-layer counts. Signed densities remain available for the original subset only (Appendix D.1); near-zero concentration should not be confused with exact zeros.

4.5 CROSS-SIZE RECURRENCE OF BROAD-PRESSURE RECIPES

At $\kappa = 0.5$, seven-site + OL1(all) has both lower validation loss and greater model-wide sparsity than four-site + OL1(all) at all three model sizes (Figure 6). The same ordering does not hold at $\kappa = 0$: four-site + OL1(all) is better on both axes at 14M and 70M, whereas seven-site + OL1(all) is better on both axes at 410M. Thus, the cross-size result is recurrence of the **high-threshold complete-recipe relationship**, not transfer of the entire threshold response. The larger cohorts lack the h-only and pressure-free controls needed to isolate pressure’s contribution or establish h-only transfer. Exact matched seven-site minus four-site OL1(all) contrasts at all five thresholds are reported in Appendix Table 8.

Raw $\mathcal{S}_{model}$ must be interpreted relative to the computation reachable by each architecture. The seven-site sparsity ceiling increases from 29.95% at 14M to 49.42% at 70M and 87.25% at 410M, largely because the dense output projection accounts for a smaller fraction of the counted products as model width and depth increase. At $\kappa = 0.5$, seven-site + OL1(all) reaches 27.48%, 40.60%, and 80.62% $\mathcal{S}_{model}$, corresponding to **91.8%, 82.2%, and 92.4% of the respective seven-site ceilings**. The much larger raw sparsity at 410M therefore reflects in part a larger architectural opportunity for the selected intervention sites, rather than an isolated effect of model size on learned zero fractions.

Absolute validation loss is not monotonic with model size under the realized protocol: the A0 endpoints are 5.2086, 4.0998, and 4.5475 for 14M, 70M, and 410M. This experiment holds the input-token budget fixed rather than tokens per parameter; the resulting budgets are 106.14, 21.20, and 3.68 tokens per parameter, and 410M also uses a lower peak learning rate. Appendix Figure 8

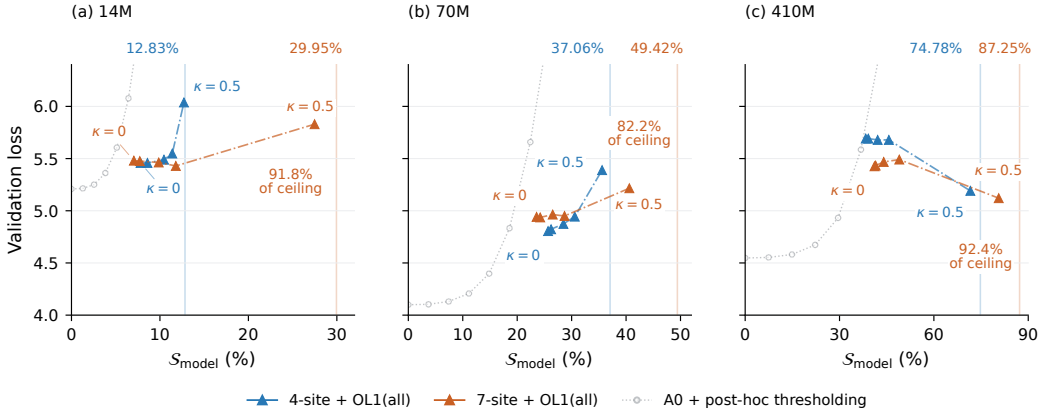


Figure 6: **The high-threshold seven-site recipe remains better on both quality and model-wide sparsity across model sizes.** Absolute validation loss versus model-wide sparsity S_{model} for Pythia-14M, 70M, and 410M. Blue and orange curves connect separately trained four-site + OL1(all) and seven-site + OL1(all) checkpoints at $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$; the gray dotted curves apply post-hoc thresholding at a, m, h, z to the corresponding A0 checkpoints. Vertical guides mark the analytic four-site and seven-site sparsity ceilings, and annotations at the seven-site $\kappa = 0.5$ endpoints report the fraction of the seven-site ceiling reached. **At $\kappa = 0.5$, seven-site + OL1(all) has both lower validation loss and greater S_{model} than four-site + OL1(all) at all three sizes, reaching 91.8%, 82.2%, and 92.4% of the respective seven-site ceilings.** The ordering at $\kappa = 0$ changes with size. Lines connect evaluated settings rather than interpolated models; full post-hoc thresholding trajectories, including higher-loss evaluations outside the displayed range, are reported in Appendix D.3.

reports the corresponding A0 optimization trajectories. **We therefore use the size sweep to test whether within-size recipe relationships recur, not as a language-model quality scaling law.**

4.6 WHEN MODEL-WIDE SPARSITY TRANSLATES TO SPEEDUP

S_{model} measures matrix-product work exposed to zero operands, but does not by itself guarantee lower latency. Whether these opportunities become useful depends on where the zeros occur, their execution structure, and the kernel used to exploit them. We evaluate the same specialized K050 implementation across 35 Pythia-14M checkpoints, including the five historical four-site h-only models, to ask which forms of learned sparsity translate into measured acceleration. Figure 1 adds the five seven-site h-only full-model timings from a separate RTX5090 session; the matched ablations here retain the historical 35-checkpoint cohort. Those historical measurements use BF16, batch-one, uncached 2,048-token inference with full-vocabulary logits on one RTX5090. Each candidate undergoes numerical qualification against the native reference on complete validation before timing (Appendix D.4).

More logical sparsity does not necessarily mean lower optimized latency (Figure 7a). At $\kappa = 0.5$, T_4/P_4 runs at 0.4595 ms with loss 6.0380, whereas T_7/P_7 runs at 0.4734 ms with loss 5.8294. The seven-site model is slower under full K050 despite its larger S_{model} , but has lower loss. These are quality–latency trade-offs, not an equal-quality speed ranking. The native-relative association ($R^2 = 0.817$) remains descriptive and uses a different native latency for each checkpoint (Appendix D.5).

The restored T_4/P_h measurements make pressure placement relevant to this comparison: at $\kappa = 0.5$, this model runs at 0.4623 ms with loss 5.7227, both lower than T_7/P_7 despite only 10.23% model-wide sparsity. At $\kappa = 0.05$, it runs at 0.5618 ms with loss 5.1956. The seven-site h-only full-model measurements appear in Figure 1; small differences across the two physical GPU/host sessions do not establish a reliable runtime ordering.

The matched ablations distinguish where sparsity is profitable. Higher thresholds produce substantial gains from projection skipping, and OL1(all) further increases this benefit for the four-site recipe at positive thresholds (Appendix D.5). Without pressure, four-site and seven-site recipes have simi-

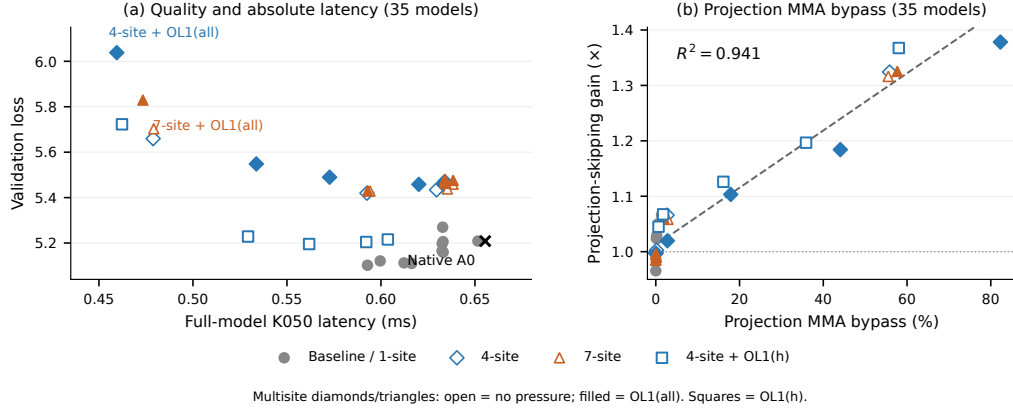


Figure 7: **The runtime value of zeros depends on their execution structure.** (a) Validation loss versus absolute full-model K050 latency for 35 qualified checkpoints; the black cross is native A0. Gray circles show A0 and one-site conditions, blue diamonds four-site recipes, and orange triangles seven-site recipes. These multisite markers are open without pressure and filled with OL1(all); blue squares add four-site OL1(h). This historical panel excludes the seven-site OL1(h) timings shown in Figure 1. (b) Projection MMA bypass for the same 35 checkpoints versus matched gain from enabling projection skipping: all-skips-off latency divided by projection-enabled, attention-dense latency. The dashed OLS fit is descriptive ($R^2 = 0.941$). MMA bypass includes replacement by scalar work, not only eliminated arithmetic. Loss and sparsity use full-validation FP16 evaluation; BF16 timing uses 64 matched inputs on an RTX5090. Neither panel identifies a causal effect of sparsity alone.

lar projection-skipping gains at matched thresholds. At $\kappa = 0.5$, adding OL1(all) raises the four-site gain from $1.324\times$ to $1.378\times$, but the seven-site gain only from $1.316\times$ to $1.325\times$, despite its much larger increase in $\mathcal{S}_{\text{model}}$. Much of that additional seven-site sparsity lies in QK/PV attention products (Figure 5), outside the projection path. Additional model-wide sparsity therefore need not bring a proportional projection-skipping benefit.

Scalar sparsity also does not fully determine the projection benefit. The kernel exploits zero structure at the level of matrix-multiply fragments, with very sparse rows sometimes handled by scalar operations instead. We call the resulting omitted matrix multiply-accumulate instructions *MMA bypasses*; a bypass need not eliminate all corresponding arithmetic. Across the cohort, projection MMA bypass is more strongly associated with projection-skipping gain ($R^2 = 0.941$) than scalar projection sparsity alone ($R^2 = 0.425$). The useful execution unit is coarser than an individual zero activation. Appendix D.5 gives the predicates, counters, and matched latencies.

The attention path provides the complementary result. Seven-site interventions expose many zero-operand QK and PV products, but the tested attention-skipping path remains slower than its dense counterpart across the cohort, even at the checkpoint with substantial additional MMA skipping. **For these 14M measurements, high-threshold projection skipping is profitable, whereas attention skipping is not.** This is a property of the measured kernel, shapes, and hardware, rather than evidence that attention sparsity is intrinsically unprofitable.

Native-relative speedup is not a pure measure of sparse-path savings: it divides native latency by optimized K050 latency, and the native cost itself differs across intervention families. Seven-site recipes can report larger native-relative speedups even when their absolute K050 latency exceeds that of the corresponding four-site model. Appendix D.5 therefore reports native, optimized, and matched-ablation latencies separately.

We used a human-guided GPT-6-Astra coding-agent loop to develop and numerically qualify K050 (OpenAI, n.d.; Luo et al., 2026). Search history and upstream-kernel compatibility remain in Appendix D.4; this is an implementation tool, not an agent-versus-human development comparison.

Together, these results separate sparsity opportunity from sparsity realization. $\mathcal{S}_{\text{model}}$ captures computation logically exposed to zeros and tracks end-to-end performance across the trained models, but the runtime value of those zeros depends on their location and execution structure. K050 profitably exploits high-threshold projection sparsity, with pressure amplifying the four-site benefit, while the additional attention sparsity remains overhead-limited in the tested implementation.

5 DISCUSSION AND CONCLUSION

Where zeros are exposed need not equal where sparsity pressure is applied. In our 14M sweep, threshold scope and pressure scope define distinct operating regimes: h-only pressure is favorable at moderate thresholds, whereas broad seven-site pressure exposes the highest sparsity at $\kappa = 0.5$. Holding h-only pressure fixed gives a cleaner threshold-placement comparison, with the broader threshold scope adding 6.44 pp sparsity for +0.0094 loss.

These observations suggest a scoped rule of thumb: in our 14M sweep, use threshold placement to expose computational reach; start pressure narrowly at the FFN hidden site when quality preservation matters, and broaden it when the target operating point requires extreme sparsity. This is not yet a recommendation across model sizes. The larger cohorts establish only the recurrence of the high-threshold broad-pressure recipe ordering.

Product accounting and kernel timing answer different questions. $\mathcal{S}_{\text{model}}$ measures how much matrix-product work is logically exposed to zero operands; the kernel study asks how much of that opportunity can be profitably realized. The distinction matters across sites: high-threshold projection sparsity produces structured work that K050 can skip with a net latency benefit, while additional Q/K/V sparsity increases the logical opportunity without producing profitable attention skipping at the tested 14M shapes. The computational value of a sparse representation therefore depends not only on how many zeros it contains, but also on where they occur and the granularity at which the implementation can exploit them.

Cross-size comparisons use a common token budget rather than equal tokens per parameter, and 410M also uses a lower peak learning rate; the A0 optimization trajectories in Appendix Figure 8 show that the three sizes finish in different realized optimization regimes. Accordingly, the scale experiment tests recurrence of within-size recipe relationships rather than quality scaling with parameter count. These conclusions concern MiniPile pretraining and complete recipes at the larger sizes. The distribution diagnostics cover 14M and describe responses rather than isolating a causal pathway for the nonlocal response. The kernel results use RTX5090 hardware, full-sequence inference, native SDPA and matched fusion controls; Figure 1 adds seven-site h-only timings from a separate physical GPU/host session. Transfer to cached decoding, larger models and other hardware remains untested.

The pressure-scope controls sharpen identification but do not complete it. Each condition has one training seed, and the shared validation set is not an independent confirmation set. Because T_7/P_4 is absent, the $P_h \rightarrow P_7$ contrast jointly changes pressure at a, m, z, q, k, v and the objective normalization; it cannot isolate Q/K/V pressure. H-only scaling remains outside the larger-size cohorts reported here. The seven-site h-only full-model timings in Figure 1 do not include matched skipping ablations. Present larger-scale conclusions concern complete broad-pressure recipes only.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.

- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- OpenAI. GPT-6 Astra. OpenAI API documentation, n.d. URL <https://developers.openai.com/api/docs/models/gpt-6-astra>. Accessed September 10, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 THRESHOLDING NONLINEARITIES

For finite $\kappa \geq 0$, both thresholding nonlinearities retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. These nonlinearities use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 ACTIVATION PRESSURE AND OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after the non-linearity where one is applied. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov & Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For diagnostics, we define the task-relative opposing component

$$\rho_{\text{opp}} = \frac{\max(0, -c)}{q + \epsilon}.$$

When projection is active, this equals the norm of the component removed from w , divided by $\|u\|_2$; it is zero when $c \geq 0$. All reported steps satisfy the guard $q > \epsilon$. It is therefore not the fraction of the pressure norm removed.

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters. We report the norm budget as active when the implemented scale satisfies $s < 1$.

Ignoring numerical stabilizers, the correction norm before the learning rate is $\min\{\lambda \|\tilde{w}\|_2, b\|u\|_2\}$. For a fixed optimizer state, once the cap binds, further increases in λ leave this ideal correction unchanged.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B SPARSITY ACCOUNTING

B.1 EXACT COUNTERS

Activation sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \varepsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 SPARSITY CEILING AND NORMALIZATION

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of 100% activation sparsity at the selected sites, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{N})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA SITES AND NONLINEARITIES

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 3 identifies the actual operands recorded after the selected nonlinearity. The same site mapping is applied in every layer, with one common threshold per condition. Figure 2 shows the shared block; Table 1 gives its thresholding and pressure assignments.

At h , the selected nonlinearity replaces GELU. Other sites retain their original activations unless a thresholding nonlinearity is applied. At zero threshold, A7's additional symmetric query/key/value nonlinearities are exact identities in both values and gradients. They therefore do not change A4's

Table 3: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

mathematical forward map in the absence of pressure. A7-OL1(all) still differs from A4-OL1(all) in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-nonlinearity context z , rather than inferring its zeros from the value mask.

C.2 ARCHITECTURE-SPECIFIC COUNTS AND SPARSITY CEILINGS

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV . Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (4)$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{N})$ be the union of operations reachable when the selected sites have 100% activation sparsity: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK ; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{N})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. For example, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros.

C.3 EVALUATION SCOPE

The original 54 endpoints pair quality and logical counts in the same eager-attention evaluation. For the ten h-only endpoints, we retain the author-approved ordinary final-validation losses and separately evaluated logical counts from the same checkpoint. Both passes cover the complete validation set; across the 30 multisite 14M endpoints, their largest absolute loss difference is 0.000106 nats. Both loss versions and uniform-pass contrasts are retained in the supplementary pressure-placement data. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

is count-pooled. This workload uses no key-value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

C.4 REALIZED TRAINING PROTOCOL

We use MiniPile and the Pythia-14M-deduped tokenizer for every size. Each document is encoded without added special tokens, followed by one end-of-sequence token. Concatenated tokens are packed into nonoverlapping 2,048-token blocks. Training uses a seeded permutation of the block indices; validation retains source order. The training split contains 1,000,000 documents and 1,491,711,416 tokens, yielding 728,374 complete blocks and an excluded 1,464-token tail. The supplementary `protocol.json` records immutable dataset and tokenizer revisions, cache hashes and implementation identities.

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234, with one training seed per condition. Parameter sweeps measure intervention variability rather than variation over independent training seeds. All quality comparisons reuse the same validation split, without a separate confirmation set. Within a size, the conditions share the initial parameter hash and training schedule. The 40-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds for each of T_4/P_0 , T_4/P_h , T_4/P_4 , T_7/P_0 , T_7/P_h , and T_7/P_7 . Each larger cohort contains A0, A1-H, and the five-threshold T_4/P_4 and T_7/P_7 sweeps. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure at h only or at all thresholded sites. Every OL1 condition uses trust budget 1 and projection stabilizer $\epsilon = 10^{-12}$. The historical four-site h-only conditions are identified by their audited executed pressure capture, not their original all-site declaration. They are valid T_4/P_h conditions; the ten h-only endpoints match the initialization, order and final-checkpoint protocol of the original cohort.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 4 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

Table 4: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

Figure 8 shows the A0 optimization trajectories under the common token budget. Training loss trends downward across the runs, but the three sizes finish in different optimization regimes: 70M reaches the lowest training loss under the realized schedule, while 410M finishes above it and exhibits a larger pre-clipping task-gradient norm late in training. These observations are **consistent with different degrees of optimization under the fixed-token protocol**, but they do not establish whether any run has or has not converged. The trajectories therefore provide context for the non-monotonic A0 validation losses in Figure 6 rather than a basis for extrapolating performance with additional training.

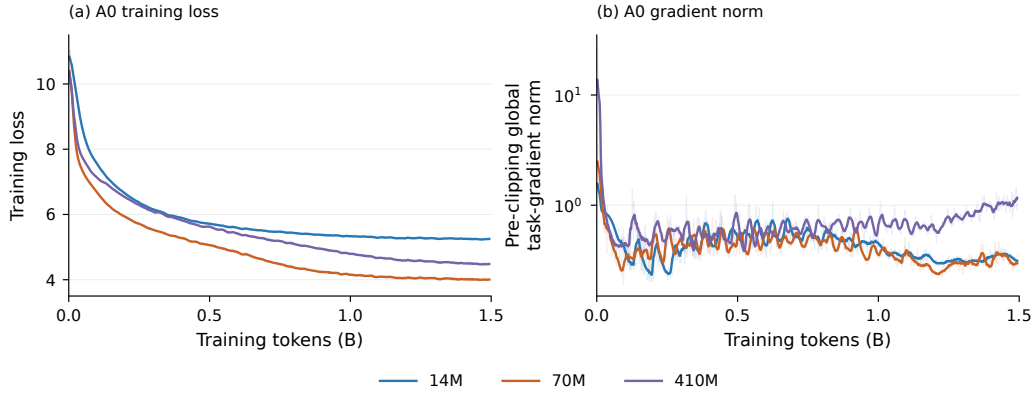


Figure 8: **A0 optimization trajectories under the common training-token budget.** (a) Training loss as a function of consumed input tokens for the 14M, 70M, and 410M A0 runs. (b) Pre-clipping global task-gradient norm over the same token budget; the logarithmic y-axis shows the large initial gradient transient and subsequent training regime. Faint traces show all 712 logged optimizer steps per size; opaque curves show a centered nine-step arithmetic moving mean, with partial windows at the edges. Norms are measured after loss unscaling and gradient accumulation, before clipping; smoothing is applied in the original units. All three sizes receive approximately 1.493B input tokens, but this corresponds to progressively fewer tokens per parameter as model size increases; the 410M run also uses a lower peak learning rate. Global norms are not normalized by parameter count. These trajectories characterize the realized optimization regimes and are not used as a convergence criterion.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

The following tables report all 64 trained endpoints, the 29 original paired 14M contrasts, 25 additional pressure-scope contrasts, and all 15 cross-scale broad-pressure recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common A7 sparsity-ceiling denominator from Figure 6, rather than a different denominator for each recipe. Here $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\max}(\text{A7})$ denotes this common-reference ratio, retained for the complete endpoint table.

Table 5: All 64 trained endpoints. T_i/P_j specifies threshold and pressure scope; P_0 means no pressure. U_{arch} uses the common seven-site ceiling within each size. H-only losses use ordinary final validation; other losses use the logical diagnostic pass (Appendix C.3). Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	T_4/P_0	$\kappa = 0$	5.4705	7.212	24.078
14M	T_4/P_0	$\kappa = 0.01$	5.4665	7.414	24.752
14M	T_4/P_0	$\kappa = 0.05$	5.4341	8.206	27.396
14M	T_4/P_0	$\kappa = 0.1$	5.4197	8.953	29.891
14M	T_4/P_0	$\kappa = 0.5$	5.6597	10.216	34.106
14M	T_4/P_h	$\kappa = 0$	5.2157	8.409	28.076

Table 5 (continued)

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	T_4/P_h	$\kappa = 0.01$	5.2045	8.586	28.665
14M	T_4/P_h	$\kappa = 0.05$	5.1956	9.036	30.167
14M	T_4/P_h	$\kappa = 0.1$	5.2287	9.343	31.194
14M	T_4/P_h	$\kappa = 0.5$	5.7227	10.227	34.146
14M	T_4/P_4	$\kappa = 0$	5.4583	7.810	26.075
14M	T_4/P_4	$\kappa = 0.01$	5.4583	8.587	28.668
14M	T_4/P_4	$\kappa = 0.05$	5.4898	10.456	34.910
14M	T_4/P_4	$\kappa = 0.1$	5.5483	11.394	38.041
14M	T_4/P_4	$\kappa = 0.5$	6.0380	12.713	42.446
14M	T_7/P_0	$\kappa = 0$	5.4684	7.218	24.097
14M	T_7/P_0	$\kappa = 0.01$	5.4588	7.618	25.433
14M	T_7/P_0	$\kappa = 0.05$	5.4379	9.127	30.471
14M	T_7/P_0	$\kappa = 0.1$	5.4287	10.425	34.805
14M	T_7/P_0	$\kappa = 0.5$	5.7029	15.387	51.371
14M	T_7/P_h	$\kappa = 0$	5.2198	8.399	28.042
14M	T_7/P_h	$\kappa = 0.01$	5.1981	8.856	29.567
14M	T_7/P_h	$\kappa = 0.05$	5.1950	10.126	33.807
14M	T_7/P_h	$\kappa = 0.1$	5.2374	10.904	36.403
14M	T_7/P_h	$\kappa = 0.5$	5.7320	16.664	55.634
14M	T_7/P_7	$\kappa = 0$	5.4802	7.054	23.551
14M	T_7/P_7	$\kappa = 0.01$	5.4758	7.723	25.786
14M	T_7/P_7	$\kappa = 0.05$	5.4628	9.863	32.929
14M	T_7/P_7	$\kappa = 0.1$	5.4295	11.797	39.385
14M	T_7/P_7	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	T_4/P_4	$\kappa = 0$	4.8054	25.673	51.944
70M	T_4/P_4	$\kappa = 0.01$	4.8229	26.224	53.060
70M	T_4/P_4	$\kappa = 0.05$	4.8742	28.490	57.644
70M	T_4/P_4	$\kappa = 0.1$	4.9439	30.574	61.861
70M	T_4/P_4	$\kappa = 0.5$	5.3895	35.596	72.022
70M	T_7/P_7	$\kappa = 0$	4.9412	23.562	47.674
70M	T_7/P_7	$\kappa = 0.01$	4.9364	24.226	49.017
70M	T_7/P_7	$\kappa = 0.05$	4.9637	26.524	53.666
70M	T_7/P_7	$\kappa = 0.1$	4.9501	28.694	58.056
70M	T_7/P_7	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	T_4/P_4	$\kappa = 0$	5.6921	38.339	43.944
410M	T_4/P_4	$\kappa = 0.01$	5.6914	39.149	44.873
410M	T_4/P_4	$\kappa = 0.05$	5.6789	42.125	48.283
410M	T_4/P_4	$\kappa = 0.1$	5.6789	45.688	52.367
410M	T_4/P_4	$\kappa = 0.5$	5.1910	71.591	82.058
410M	T_7/P_7	$\kappa = 0$	5.4263	41.121	47.133
410M	T_7/P_7	$\kappa = 0.01$	5.4331	41.478	47.542
410M	T_7/P_7	$\kappa = 0.05$	5.4679	44.053	50.494
410M	T_7/P_7	$\kappa = 0.1$	5.4907	48.998	56.161
410M	T_7/P_7	$\kappa = 0.5$	5.1207	80.616	92.401

D.1 ACTIVATION SPARSITY AND SIGNED-DENSITY COVERAGE

The seven frozen-checkpoint evaluations in Figure 11 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 9 reports the zero mass and

Table 6: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1(all) – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1(all) – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1(all) – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1(all) – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1(all) – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1(all) – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1(all) – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1(all) – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1(all) – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1(all) – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 7: Pressure-scope increments and threshold-placement contrasts at 14M. Each row subtracts the reference at the same κ . These are single-seed comparisons.

Treatment – reference	κ	ΔL	$\Delta\mathcal{S}_{\text{model}}$ (pp)
$T_4/P_h - T_4/P_0$	0	-0.2548	+1.197
$T_4/P_h - T_4/P_0$	0.01	-0.2620	+1.172
$T_4/P_h - T_4/P_0$	0.05	-0.2385	+0.830
$T_4/P_h - T_4/P_0$	0.1	-0.1910	+0.390
$T_4/P_h - T_4/P_0$	0.5	+0.0630	+0.012
$T_4/P_4 - T_4/P_h$	0	+0.2425	-0.599
$T_4/P_4 - T_4/P_h$	0.01	+0.2538	+0.001
$T_4/P_4 - T_4/P_h$	0.05	+0.2942	+1.421
$T_4/P_4 - T_4/P_h$	0.1	+0.3196	+2.051
$T_4/P_4 - T_4/P_h$	0.5	+0.3153	+2.486
$T_7/P_h - T_7/P_0$	0	-0.2486	+1.182
$T_7/P_h - T_7/P_0$	0.01	-0.2607	+1.238
$T_7/P_h - T_7/P_0$	0.05	-0.2429	+0.999
$T_7/P_h - T_7/P_0$	0.1	-0.1913	+0.479
$T_7/P_h - T_7/P_0$	0.5	+0.0292	+1.277
$T_7/P_7 - T_7/P_h$	0	+0.2604	-1.345
$T_7/P_7 - T_7/P_h$	0.01	+0.2777	-1.133
$T_7/P_7 - T_7/P_h$	0.05	+0.2678	-0.263
$T_7/P_7 - T_7/P_h$	0.1	+0.1921	+0.893
$T_7/P_7 - T_7/P_h$	0.5	+0.0974	+10.819
$T_7/P_h - T_4/P_h$	0	+0.0041	-0.010
$T_7/P_h - T_4/P_h$	0.01	-0.0064	+0.270
$T_7/P_h - T_4/P_h$	0.05	-0.0006	+1.090
$T_7/P_h - T_4/P_h$	0.1	+0.0087	+1.560
$T_7/P_h - T_4/P_h$	0.5	+0.0094	+6.436

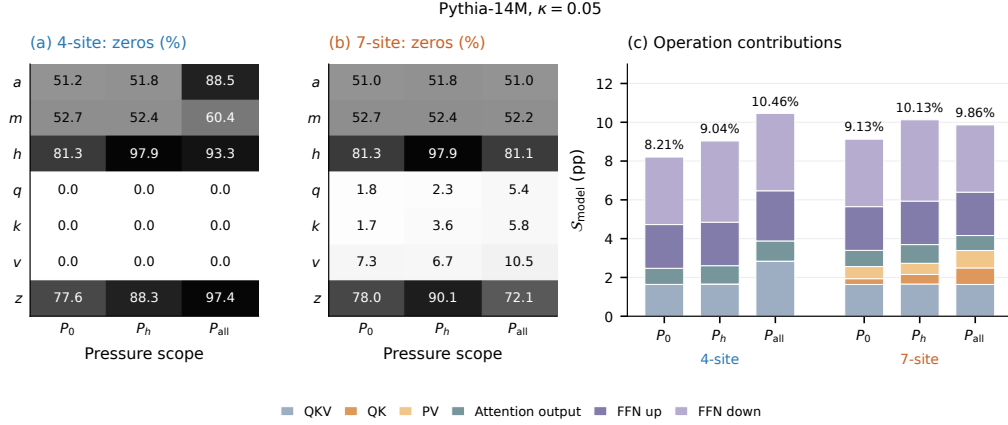


Figure 9: **Pressure-scope effects at the moderate threshold $\kappa = 0.05$.** The six 14M recipes use the same count pooling and operation denominator as Figure 5. H-only pressure concentrates zeros at h while preserving a different pattern at m and Q/K/V from broad pressure. Rounded percentages do not establish all-zero or fully dense tensors.

the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention in Figure 11. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%. These historical densities cover no h-only multisite checkpoints. Figure 5 instead uses retained final-validation site counters for all six threshold/pressure combinations, separating m and h . Its exact-zero counts were measured independently of the signed histogram reruns; small pass-to-pass rounding differences remain visible.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional thresholding. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 8: All matched A7-OL1(all) minus A4-OL1(all) comparisons. Negative loss changes and positive sparsity changes favor A7-OL1(all).

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

Table 9: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1(all)	0	FFN	67.22	0.0003	0.0000
A4-OL1(all)	0	Attention	< 0.01	5.1717	0.6162
A4-OL1(all)	0.05	FFN	86.71	0.0003	0.0000
A4-OL1(all)	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1(all)	0.5	FFN	99.31	0.0001	0.0000
A4-OL1(all)	0.5	Attention	0.22	0.0003	0.0000
A7-OL1(all)	0	FFN	64.52	0.0063	0.0000
A7-OL1(all)	0	Attention	< 0.01	5.4657	0.4648
A7-OL1(all)	0.05	FFN	75.29	0.0106	0.0000
A7-OL1(all)	0.05	Attention	7.26	6.6719	0.6932
A7-OL1(all)	0.5	FFN	93.64	0.0037	0.0000
A7-OL1(all)	0.5	Attention	95.60	0.2396	0.0009

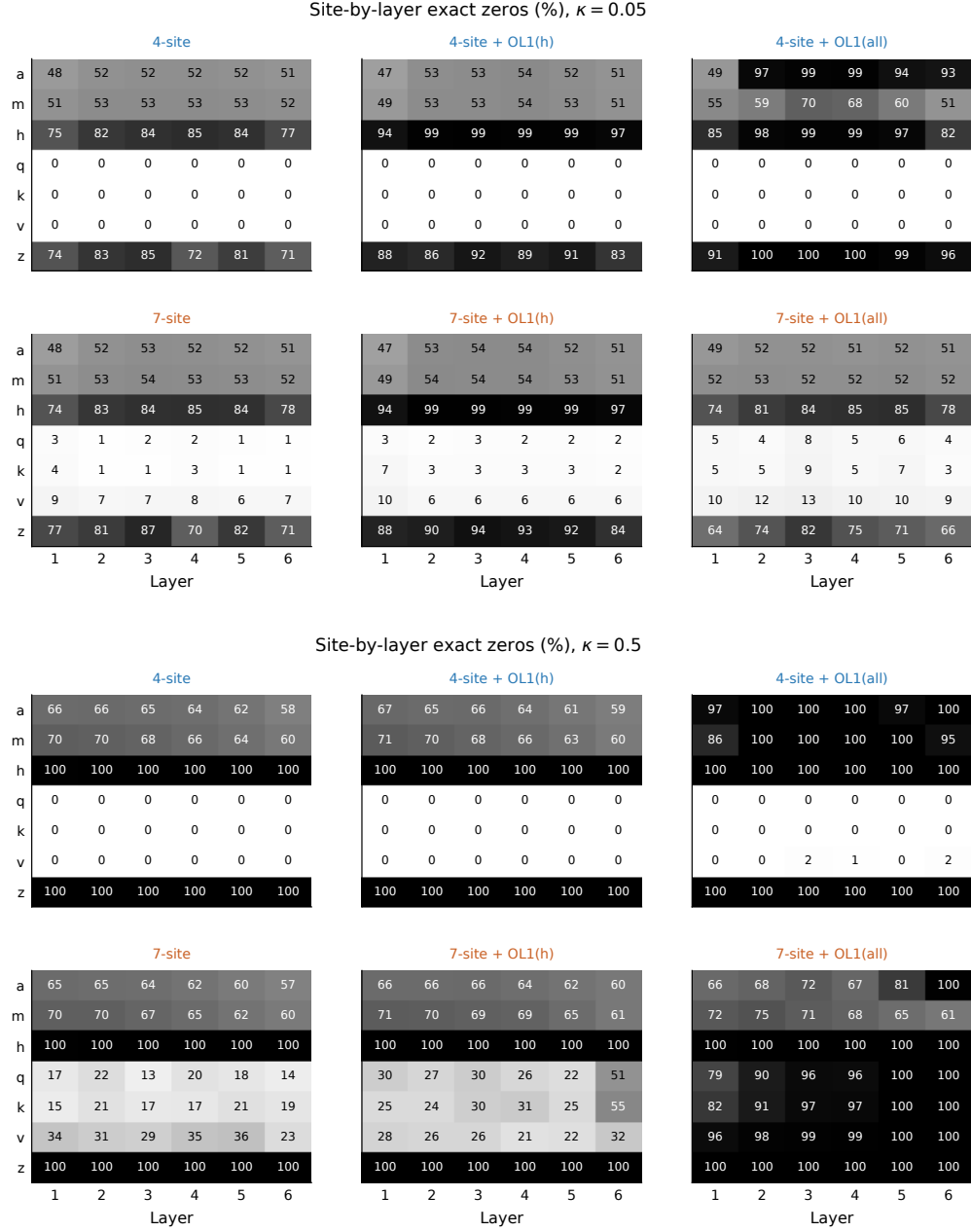


Figure 10: **Site-by-layer zero fractions under matched pressure scopes.** The six recipes at $\kappa = 0.05$ (top) and 0.5 (bottom). Each cell pools integer counts over all 338 validation blocks at one site and layer. Q/K are post-RoPE; values are rounded to whole percentages, so a displayed 100 does not imply an all-zero tensor. The complete five-threshold counters and source identities accompany the supplementary data.

How interventions reshape activation distributions (Pythia-14M)

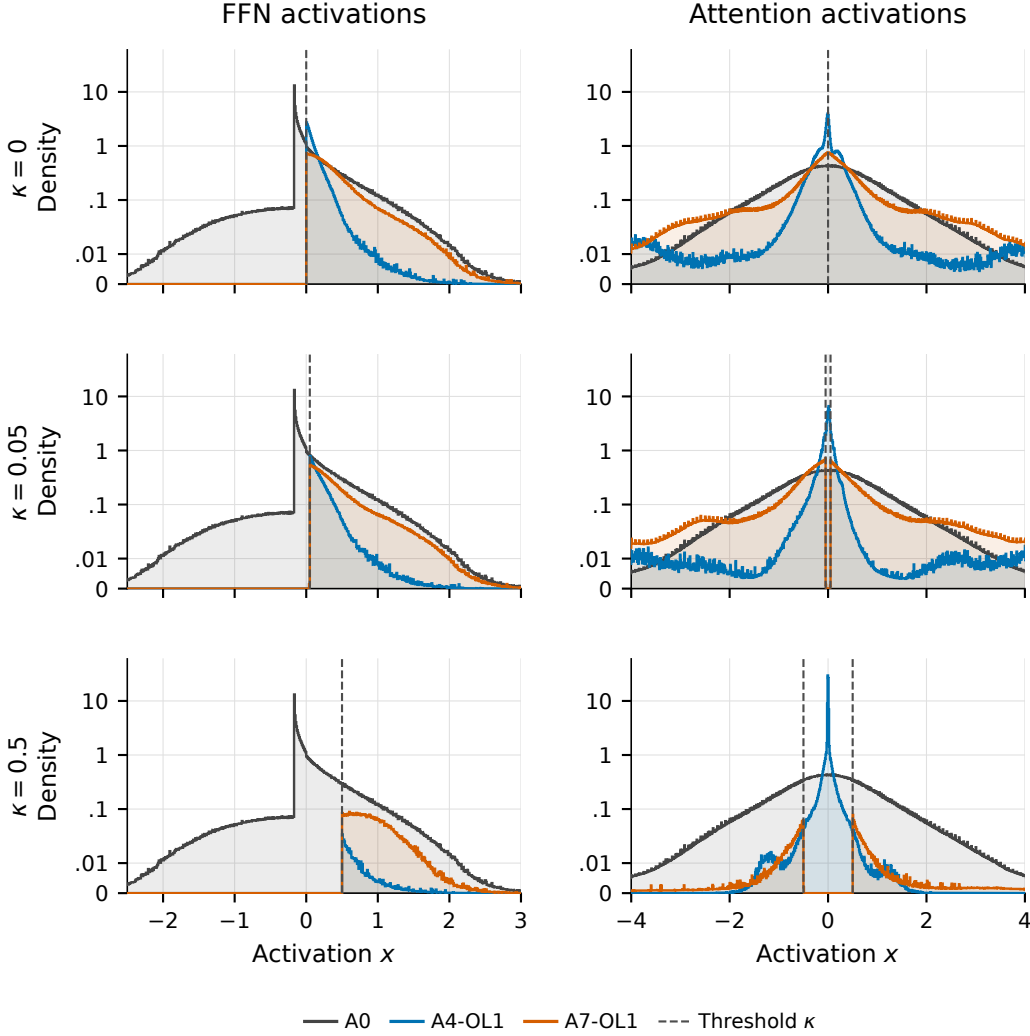


Figure 11: **Interventions reshape FFN and attention distributions differently. Exact zeros are omitted from the curves and reported in Table 9.** Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01; filled areas on this scale do not represent probability mass. Dashed lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc thresholding.

D.2 OPERATION CONTRIBUTIONS TO MODEL-WIDE SPARSITY

An operation’s contribution to S_{model} is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 10 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1(all) to A7-OL1(all) adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation sparsity need not yield an equal change in model-wide sparsity. Figure 12 displays the operation-level view at the same threshold and all three sizes.

Table 10: Contributions to $100S_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1(all)	A7-OL1(all)	A4-OL1(all)	A7-OL1(all)	A4-OL1(all)	A7-OL1(all)
QKV	3.173	2.430	8.820	6.231	17.563	16.992
QK	0.000	8.324	0.000	5.052	0.000	5.206
PV	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

Operation contributions at $\kappa = .5$

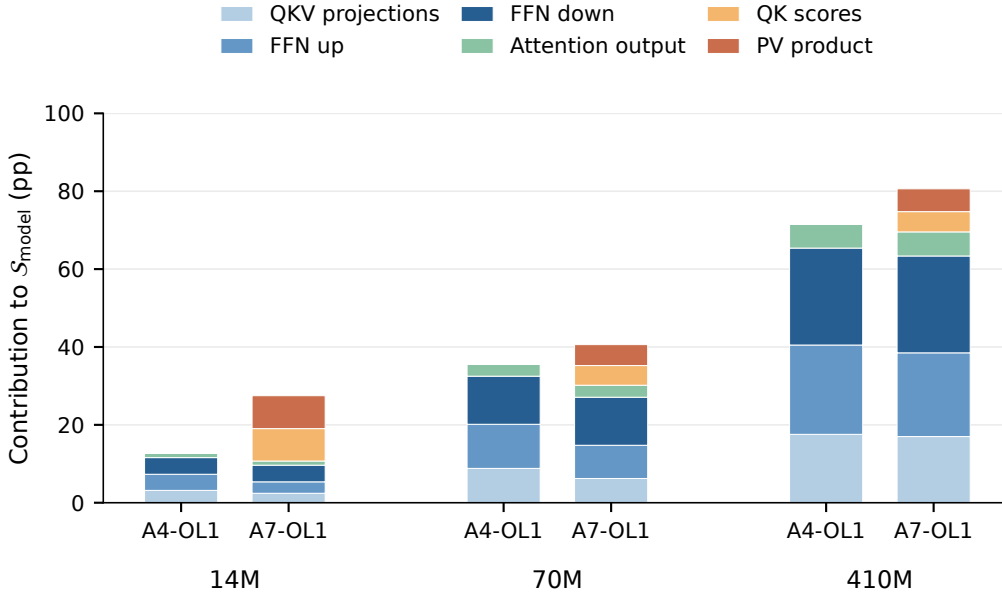


Figure 12: Operation accounting for A4-OL1(all) and A7-OL1(all) at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1(all), outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.3 COMPLETE HISTORICAL POST-HOC THRESHOLDING TRAJECTORIES

Historical plot legends retain “clipping” for post-hoc thresholding and “A4-OL1/A7-OL1” for the all-site pressure recipes. They do not include the newly integrated h-only conditions. The historical post-hoc release contains 540 evaluations: ten thresholding targets for each of the original 54 trained checkpoints. The ten newly included h-only endpoints have no corresponding post-hoc sweep. The release joins 350 evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 13–15 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

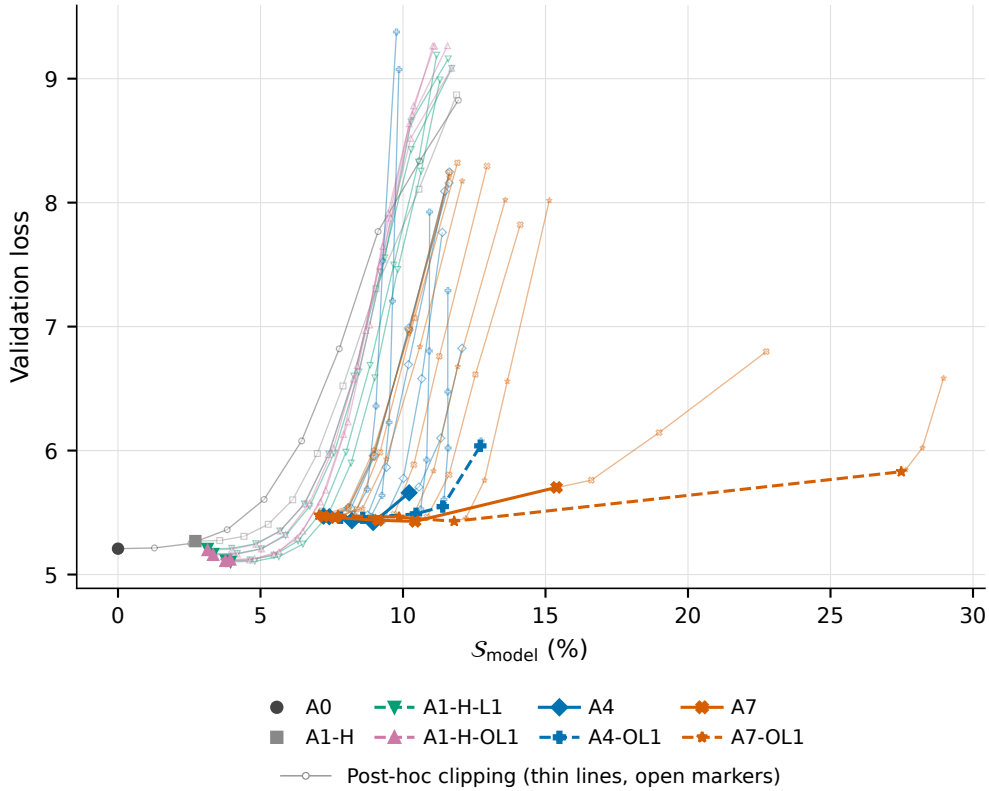


Figure 13: Complete 14M training and post-hoc thresholding results: 30 trained endpoints and 300 post-hoc thresholding evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

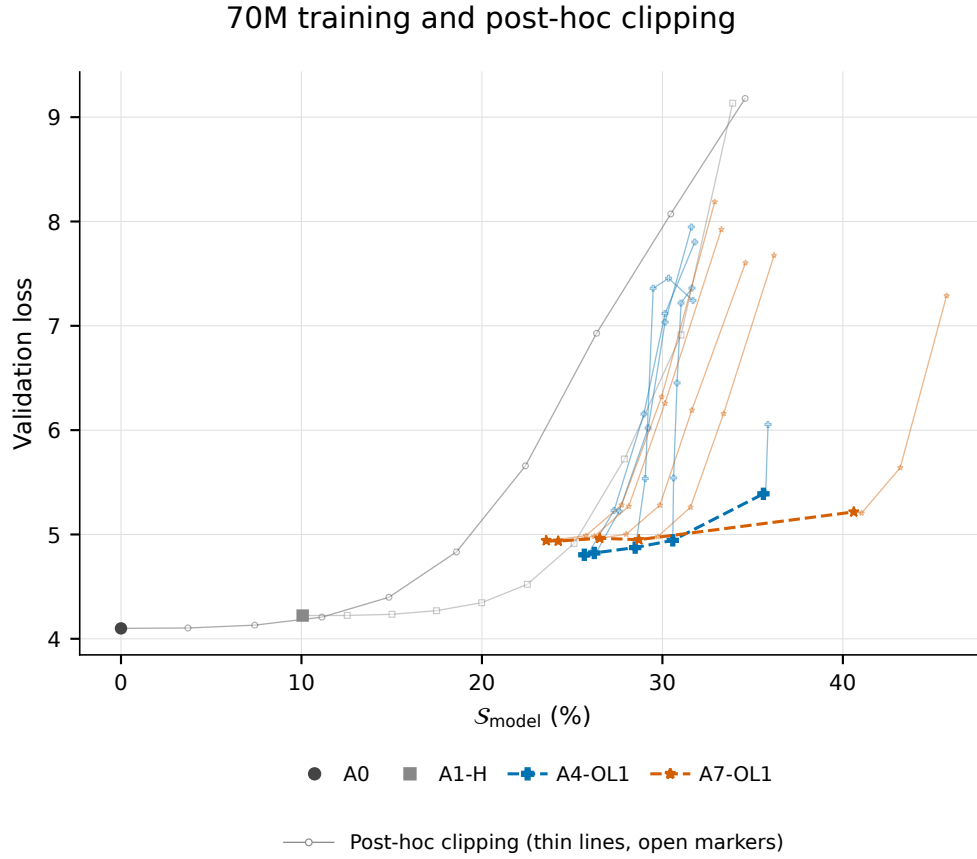


Figure 14: Complete 70M training and post-hoc thresholding results: 12 trained endpoints and 120 post-hoc thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing thresholding targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 13.

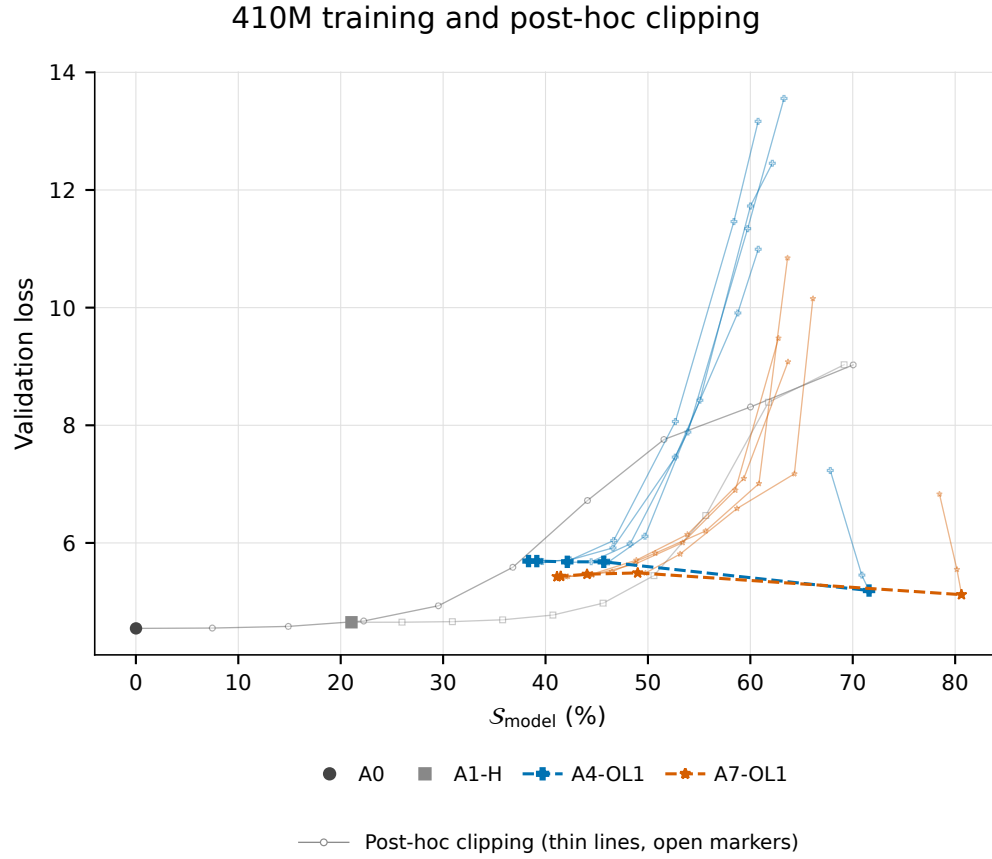


Figure 15: Complete 410M training and post-hoc thresholding results: 12 trained endpoints and 120 post-hoc thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing thresholding targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 13.

D.4 KERNEL QUALIFICATION AND ABLATIONS

The unchanged Sakana/TwELL control achieves $1.3001\times$ on its supported SparseLM-0.5B/H100 workload. Its raw projection primitive at the Pythia-14M width $N = 128$ instead gives relative L2 error 0.69055, exceeding the 0.02 bound; that test stops before full-model Pythia timing. This is a compatibility test, not an unchanged upstream speedup comparison on our model. The later P0 adapter changes signed-value packing, shape tails and numerical handling for Pythia. It retains dense attention and uses native projection fallback on A0. Table 11 reports this adapted baseline.

Historical proposals are replayed on four representative checkpoints; the incumbent in Figure 16 uses fixed A7-OL1(all), $\kappa = 0.5$. It reaches $1.6024\times$ by iteration 11 and $1.7830\times$ at iteration 42. Final implementation and ablation summaries retain their original 30-checkpoint cohort. The main absolute-latency panel additionally restores the five qualified T_4/P_h checkpoints from the same historical 35-checkpoint sweep; they implement a valid, distinct pressure scope. These are existing measurements, reaggregated from raw timing pairs with the same geometric-mean rule. Figure 1 adds five measured T_7/P_h full-model timings from a separate RTX5090 session. They do not extend the historical skipping-ablation cohort. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

Sparse kernel auto-research for full-model acceleration

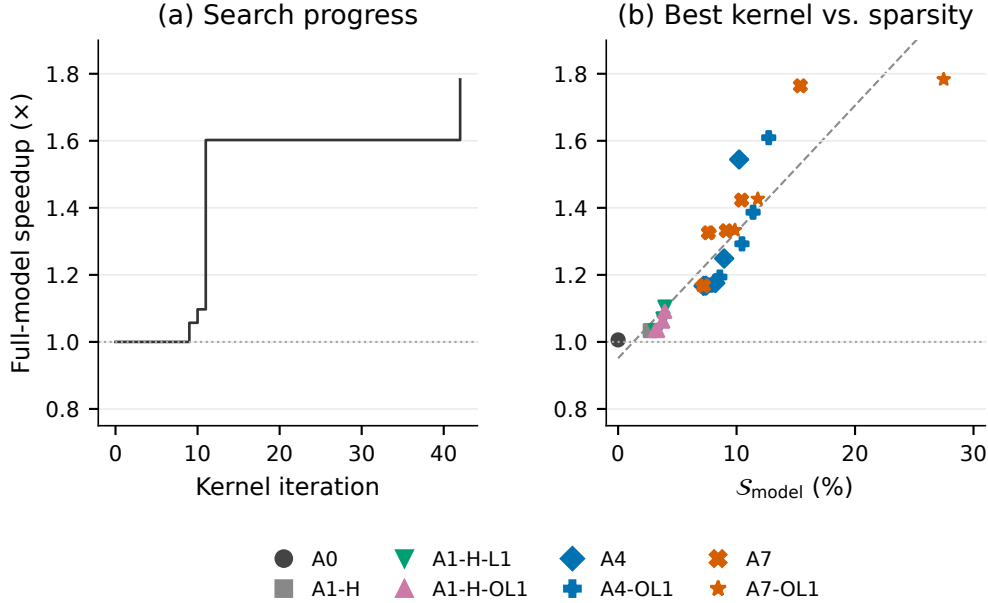


Figure 16: **Kernel-search history and the final checkpoint sweep.** (a) Best qualified incumbent across 42 eligible retrospective proposals on the fixed seven-site + OL1(all), $\kappa = 0.5$ checkpoint. (b) Final K050 speedup across the 30-checkpoint cohort; this historical view distinguishes individual recipes. Both use the same native PyTorch/SDPA CUDA-graph reference and BF16, batch-one, 2,048-token protocol on one RTX5090. Search replay and final evaluation are separate timing measurements. The history describes one human-guided development trajectory, not a comparison of coding agents.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development. Kernel measurements use Python 3.12.3, PyTorch 2.11.0+cu128, Transformers 5.12.1, CUDA 12.8 and nvcc 12.8.93. The supplementary protocol records dependency and frozen-source hashes.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here

$\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence. Failed candidates are excluded from qualified speedup summaries. P0, an early projection adapter derived from the released kernels of Cetin et al. (2026), qualifies on only four checkpoints. Its average is not comparable to a 30-checkpoint mean or an unchanged upstream benchmark.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. K049 is its searched predecessor; K050 adds fusion of the two input LayerNorms while preserving their separate affine parameters and thresholds. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. The historical ablation summary forms ratios from separately measured, native-normalized timings. Figure 7b instead directly divides candidate latency geometric means, as defined below. K050’s sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible QK and 67.2% of PV tensor-core matrix multiply-accumulate (MMA) instructions, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

The attention path tests whether either warp-distributed operand fragment contains only zeros, and issues an MMA only when both fragments contain a nonzero. Partial sparsity can therefore leave an entire instruction active. Zero detection, warp votes and branches recur inside the kernel; causal masking and softmax still execute. A zero QK score does not imply a zero softmax probability. The measurements do not identify a profitable joint-zero pattern or break-even threshold. They show that the skipped work does not repay these costs for the measured 14M shapes and hardware.

Table 11: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0’s four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5 STRUCTURED SPARSITY AND MATCHED RUNTIME DIAGNOSTICS

This subsection preserves the original 30-checkpoint diagnostic subset; the five restored h-only records and updated 35-point fit are reported below.

For the original diagnostic subset, we reconcile the same 30 checkpoints with their canonical logical-product counts, retained kernel diagnostics, and raw matched timing pairs. Table 12 separates native and optimized latencies for each checkpoint. Let t_N, t_0, t_P, t_K denote the geometric-mean latencies of native execution, all-skips-off K050, projection-only K050, and full K050. The gains from enabling projection and attention skipping are t_0/t_P and t_P/t_K , respectively. Both controls retain K050’s fusion; the former disables attention skipping in both implementations. These are full-model latency ratios, not timings of the projection operations alone. t_N/t_0 compares native execution with the optimized all-skips-off control, which also changes dense kernels and layouts, so it does not isolate fusion. Native-relative full speedup is t_N/t_K ; t_N uses the native measurements paired with

full K050. Each geometric mean uses 64 inputs, seven paired passes, and three processes, matching the original timing protocol.

Execution units. QKV and FFN-up omit a $16 \times 8 \times 16$ MMA when the entire 16-token by 16-feature activation fragment is zero. Their counters are reconstructed from the actual BF16 operands using the implemented predicate. FFN-down and attention output process eight real token rows per padded 16-row MMA. Rows with at most two nonzeros may use scalar operations under the kernel’s numerical safety checks; remaining empty feature fragments bypass MMAs. These latter counters are instrumented and independently checked against operand-derived counts. Their bypass totals include replacement by scalar work as well as empty fragments, so they are not counts of eliminated arithmetic. We pool bypassed and eligible MMA counts across the four projection families before dividing, retaining the padding and scalar-work distinctions in the supplementary data.

Attention omits an MMA when either complete operand fragment is zero. In 29 checkpoints, QK skips 0% and PV skips 10.42% of eligible instructions; the PV baseline includes masked or padded probability work. Only seven-site + OL1(all) at $\kappa = 0.5$ has additional skipping, reaching 58.00% for QK and 67.21% for PV. Attention-skipping gain nevertheless remains below one for every checkpoint ($0.9832\text{--}0.9894\times$). These instruction counts include tiled causal padding, unlike the unmasked scalar logical-product denominator.

Associations and matched recipes. Figure 17 compares scalar and instruction-level predictors. Projection-skipping gain has Pearson $r = 0.973$, $R^2 = 0.946$ against projection MMA bypass, versus $r = 0.643$, $R^2 = 0.413$ against the projection scalar zero-product fraction. The within-family $\mathcal{S}_{\text{model}}$ versus native-relative speedup fits have $R^2 = 0.542, 0.784$, and 0.762 for the baseline/one-site, four-site, and seven-site groups (ten checkpoints each). Centering both variables within family gives $R^2 = 0.660$. The global relation is therefore not solely separation between family means; all fits remain descriptive and do not control for threshold, pressure, quality, or native cost. Attention MMA skipping has only two distinct predictor values, with one checkpoint at the larger value, limiting its correlation to that contrast.

Across all five matched OL1 thresholds, four-site recipes have greater projection scalar contribution, more projection MMA bypass, and larger projection-skipping gains than seven-site recipes. Without pressure, small bypass differences change sign, although four-site has the larger gain at each threshold. At $\kappa = 0.5$ with OL1, four-site bypasses 82.30% of projection MMAs versus 57.68% for seven-site. Its full K050 latency is also lower: 0.45948 versus 0.47337 ms. Seven-site nevertheless reports the larger native-relative speedup ($1.783\times$ versus $1.609\times$), because its native baseline is slower (0.84409 versus 0.73944 ms). This relative-speedup/absolute-latency reversal holds in all eight positive-threshold four-/seven-site pairs, with and without OL1.

The native implementation executes Q/K/V gates separately at nonzero thresholds; K050 fuses them with layout conversion and RoPE, including in the all-skips-off control. Similar all-skips-off latencies support native gate overhead as an explanation for the denominator difference, but do not isolate each operation’s timing contribution.

Coverage and unavailable quantities. Logical sparsity uses retained FP16 counts. Kernel counters use actual BF16 operands over all 338 validation blocks; timing uses the original 64-input subset. Pure-zero-only versus scalar-replacement bypass counts, counters restricted to the timed inputs, unmasked-only attention MMA counts, and isolated per-operation latency attribution are unavailable. No such values are inferred from $\mathcal{S}_{\text{model}}$. Supplementary `kernel/checkpoints.csv` retains all 30 records with operation counts, skip counters, latencies, and explicit missing fields; `kernel/associations.csv` and `kernel/matched-pairs.csv` retain the complete fits and matched contrasts.

The five restored four-site h-only records are given in Table 13. Adding them to the original 30 changes the projection-bypass R^2 from 0.946 to 0.941, and the scalar-projection R^2 from 0.413 to 0.425. Figure 7 uses all 35 qualified records; historical association panels retain their explicitly stated 30-point subset. Attention skipping remains slower than attention-dense execution for all 35. Seven-site h-only full-model timing is shown separately in Figure 1; no corresponding skipping ablations are added here.

Table 12: **Matched K050 latency ablations for the original 30-checkpoint subset.** Entries are geometric-mean full-model latencies in milliseconds. Native uses the reference paired with full K050. All-skips-off retains the optimized implementation but disables both sparse paths; projection-only enables projection skipping with attention skipping disabled. The dose is λ for local L1/OL1 and κ for four-/seven-site recipes; multisite OL1 fixes $\lambda = 1$. Ratios should be computed from the unrounded supplementary values. Differences across checkpoints are not independent-seed estimates.

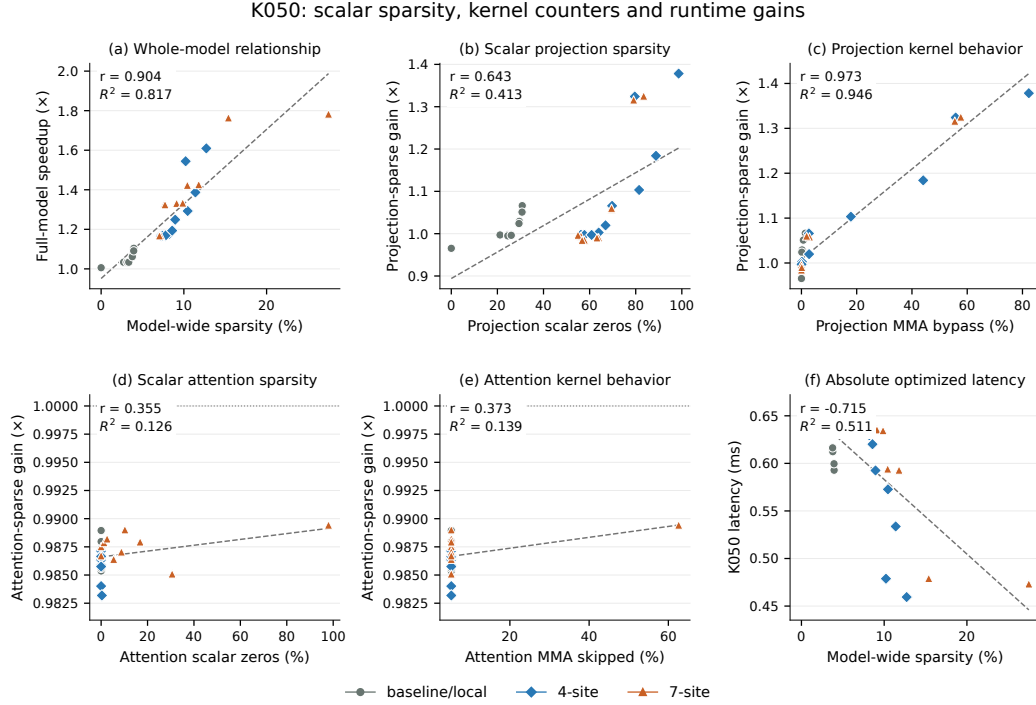
ID	Recipe	λ/κ	Native	All skips off	Proj. only	Full K050
c01	A0	–	0.65544	0.62147	0.64373	0.65157
c02	A1-H	–	0.65417	0.62230	0.62420	0.63282
c03	A1-H-L1	0.05	0.65435	0.62190	0.62375	0.63302
c04	A1-H-L1	0.1	0.65422	0.62238	0.62408	0.63251
c05	A1-H-L1	0.5	0.65447	0.62209	0.60448	0.61226
c06	A1-H-L1	1	0.65455	0.62296	0.58428	0.59282
c07	A1-H-OL1	0.05	0.65413	0.62234	0.62557	0.63256
c08	A1-H-OL1	0.1	0.65446	0.62224	0.62467	0.63308
c09	A1-H-OL1	0.5	0.65438	0.62290	0.60831	0.61633
c10	A1-H-OL1	1	0.65426	0.62181	0.59167	0.59960
c11	A4	0	0.73993	0.62327	0.62495	0.63391
c12	A4	0.01	0.74030	0.62344	0.62535	0.63391
c13	A4	0.05	0.74012	0.62313	0.62148	0.62958
c14	A4	0.1	0.73999	0.62294	0.58446	0.59255
c15	A4	0.5	0.73940	0.62389	0.47116	0.47882
c16	A4-OL1(all)	0	0.74048	0.62294	0.62479	0.63328
c17	A4-OL1(all)	0.01	0.74015	0.62394	0.61189	0.62014
c18	A4-OL1(all)	0.05	0.74025	0.62340	0.56503	0.57268
c19	A4-OL1(all)	0.1	0.74017	0.62296	0.52610	0.53371
c20	A4-OL1(all)	0.5	0.73944	0.62263	0.45175	0.45948
c21	A7	0	0.74015	0.62266	0.62605	0.63396
c22	A7	0.01	0.84616	0.62117	0.63061	0.63835
c23	A7	0.05	0.84608	0.62064	0.62669	0.63533
c24	A7	0.1	0.84553	0.62065	0.58647	0.59417
c25	A7	0.5	0.84523	0.62121	0.47200	0.47914
c26	A7-OL1(all)	0	0.74032	0.62343	0.62561	0.63403
c27	A7-OL1(all)	0.01	0.84579	0.62111	0.63089	0.63842
c28	A7-OL1(all)	0.05	0.84593	0.62147	0.62755	0.63452
c29	A7-OL1(all)	0.1	0.84583	0.62105	0.58583	0.59298
c30	A7-OL1(all)	0.5	0.84409	0.62071	0.46837	0.47337

Table 13: Restored four-site h-only runtime measurements. All entries use full-model geometric-mean latency in ms, pooled from the original matched timing pairs. These five records extend the original 30-checkpoint analysis without new execution.

κ	Loss	Native	All skips off	Projection only	Full K050
0	5.2157	0.74038	0.62311	0.59625	0.60366
0.01	5.2045	0.74027	0.62376	0.58440	0.59216
0.05	5.1956	0.74012	0.62376	0.55392	0.56180
0.1	5.2287	0.74001	0.62433	0.52171	0.52931
0.5	5.7227	0.73957	0.62339	0.45587	0.46225

D.6 EVIDENCE AND DATA ACCOMPANYING THE DRAFT

The supplementary data contain the original 54 training endpoints plus ten verified h-only endpoints, all 540 historical post-hoc evaluations, seven signed histograms, and the original 30-checkpoint kernel reduction. The pressure-placement extension adds both loss passes, site/layer counters, OL1 geometry, matched contrasts and five historical h-only runtime records. The README defines coverage and units; `SOURCES.json` records copy hashes and `protocol.json` records implementation and data identities. These files accompany the repository, not a standalone checkpoint or kernel release.



30 checkpoints. Counters: 338-block BF16 diagnostic; timings: 64-input subset. MMA bypass includes SIMT substitution and padding.

Figure 17: Scalar sparsity, skipped instructions, and matched runtime gains. The 30 checkpoints share colors and shapes by family. Projection and attention gains are t_0/t_P and t_P/t_K ; lines are descriptive OLS fits with intercepts. Projection bypass includes scalar substitution and padded MMA rows. Attention skip fractions include masked/padded work and take only two distinct values. The remaining pages of the accompanying diagnostic PDF show within-family fits, matched threshold comparisons, and absolute latency traces. Counters cover 338 validation blocks, while timings cover 64 matched inputs.